# Analisi dettagliata del DEBUG.EXE (8086, 16 bit)

## PARTE 1: I Flag dell'8086

Il registro **FLAGS** è un registro a 16 bit dove ogni bit rappresenta uno stato. DEBUG li mostra tutti insieme come stringa di sigle.

```
Bit:  15  14  13  12  11  10   9   8   7   6   5   4   3   2   1   0
       –   –   –   –  OF  DF  IF  TF  SF  ZF   –  AF   –  PF   –  CF
```

### CF – Carry Flag → `nc` / `cy`

Viene alzato quando una somma o sottrazione **supera la capacità del registro**.

```
MOV AL, 0FFh
ADD AL, 01h      ; FFh + 01h = 100h, non entra in 8 bit
                 ; → CF = 1 → DEBUG mostra "cy"
```

Nella sottrazione si alza quando si "prende in prestito" dal bit superiore (borrow).
Usato anche per operazioni a 64+ bit concatenate con `ADC` / `SBB`.

### PF – Parity Flag → `po` / `pe`

Conta i bit a **1 nel byte meno significativo** del risultato:

- **pe** (Parity Even) → numero **pari** di bit a 1
- **po** (Parity Odd) → numero **dispari** di bit a 1

```
Risultato = 07h = 0000 0111b  → tre bit a 1 → dispari → "po"
Risultato = 03h = 0000 0011b  → due  bit a 1 → pari   → "pe"
```

Oggi quasi inutilizzato, ma storicamente serviva per il **controllo di parità** nelle comunicazioni seriali.

### AF – Auxiliary Carry Flag → `na` / `ac`

Carry dal **bit 3 al bit 4**, cioè dalla cifra esadecimale bassa a quella alta.
Serve esclusivamente per le istruzioni **BCD** (Binary Coded Decimal): `DAA`, `DAS`, `AAA`, `AAS`.

```
  09h = 0000 1001b
+ 01h = 0000 0001b
__________________
  10h = 0001 0000b   ← carry dal nibble basso → AF = 1 → "ac"
```

### ZF – Zero Flag → `nz` / `zr`

Alzato quando il risultato di un'operazione è **esattamente zero**.

```
MOV AL, 05h
SUB AL, 05h      ; risultato = 0 → ZF = 1 → "zr"

CMP AL, 03h      ; 0 - 3 ≠ 0   → ZF = 0 → "nz"
```

Fondamentale nei salti condizionali: `JZ` (salta se zr), `JNZ` (salta se nz).

### SF – Sign Flag → `pl` / `ng`

Riflette il **bit più significativo** del risultato (bit 7 per byte, bit 15 per word).
In aritmetica con segno, quel bit indica il segno.

```
MOV AL, 7Fh
ADD AL, 01h      ; 7Fh + 01h = 80h = 1000 0000b
                 ; bit 7 = 1 → SF = 1 → "ng" (negativo)

MOV AL, 01h
ADD AL, 01h      ; risultato = 02h, bit 7 = 0 → "pl" (positivo)
```

### TF – Trap Flag (non mostrato direttamente)

Mette il processore in modalità **single-step**: esegue una sola istruzione poi genera un interrupt (INT 1).
È proprio quello che usa DEBUG.EXE quando premi `T` (Trace)!
Non ha una sigla nella stringa perché DEBUG lo gestisce internamente.

### IF – Interrupt Flag → `di` / `ei`

- **ei** (Enable Interrupts) → il processore risponde agli interrupt hardware (tastiera, timer, ecc.)
- **di** (Disable Interrupts) → interrupt mascherati

```
CLI   ; Clear Interrupt → "di"  (usato in sezioni critiche)
STI   ; Set Interrupt   → "ei"  (ripristino normale)
```

In un programma DOS normale vedrai sempre **ei**, perché DOS ha bisogno degli interrupt per funzionare.

### DF – Direction Flag → `up` / `dn`

Controlla la **direzione** delle istruzioni stringa (`MOVSB`, `LODSB`, `STOSB`, ecc.):

- **up** → SI/DI vengono **incrementati** dopo ogni operazione
- **dn** → SI/DI vengono **decrementati**

```
CLD   ; Clear Direction → "up"  (da indirizzo basso ad alto)
STD   ; Set Direction   → "dn"  (da indirizzo alto a basso)
```

### OF – Overflow Flag → `nv` / `ov`

Alzato quando il risultato di un'operazione **con segno** è fuori range.
Diverso dal Carry, che è per operazioni **senza segno**.

```
Signed byte: da -128 a +127


  MOV AL, 7Fh    ; +127
  ADD AL, 01h    ; +127 + 1 = +128, ma +128 non esiste in signed byte!
                 ; OF = 1 → "ov"
                 ; CF = 0 → "nc"  (nessun carry unsigned)
```

La differenza tra CF e OF è fondamentale:

| Situazione | CF | OF |
|---|---|---|
| `FFh + 01h` (255+1 unsigned) | cy | nv |
| `7Fh + 01h` (+127+1 signed) | nc | ov |
| `FFh + 01h` visto come signed (-1+1=0) | cy | nv |

## PARTE 2: La riga di disassembly

```
076A:0010   B8 6D 07
```

### Il formato completo in DEBUG

Quando usi il comando `U` (Unassemble) o esegui con `T` (Trace), DEBUG mostra:

```
SSSS:OOOO   XX XX XX   MNEMONICO OPERANDI
  ↑    ↑      ↑              ↑
  CS   IP   byte grezzi   istruzione decodificata
```

### `076A` – Il segmento CS

L'8086 usa la **segmentazione della memoria**. L'indirizzo fisico si calcola così:

```
Indirizzo fisico = Segmento × 10h + Offset


076A × 10h = 07A60h  (shift di 4 bit a sinistra)
+ 0010h
_________
= 07A70h             ← indirizzo fisico in RAM
```

Il valore `076A` dipende da **dove DOS ha caricato il tuo programma** in memoria.
Ogni esecuzione potrebbe darti un valore diverso (ma in DOSBox tende a essere costante).

### `0010` – L'offset IP

**IP (Instruction Pointer)** punta alla **prossima istruzione da eseguire**.
Vale `0010h` perché prima del codice vero c'è il **PSP (Program Segment Prefix)**: un'intestazione di 256 byte (100h) che DOS prepone ad ogni `.COM`, ma per i `.EXE` il CS parte già dopo il PSP, quindi IP parte da `0000h` e `0010h` indica che ci sono già alcune istruzioni prima.

### `B8 6D 07` – I byte grezzi (opcode)

In memoria trovi letteralmente questi tre byte:

```
B8        → opcode di  MOV AX, imm16
6D 07     → valore immediato a 16 bit in little-endian
```

**Little-endian** significa che il byte meno significativo viene prima:

```
6D 07  →  byte basso = 6Dh, byte alto = 07h
       →  valore = 076Dh
```

Quindi l'istruzione completa è:

```
MOV AX, 076Dh
```

Che nel sorgente era scritto come:

```
MOV AX, @DATA
```

`@DATA` è una **direttiva MASM** risolta a compile-time con l'indirizzo del segmento dati.
MASM ha calcolato che `.DATA` finirà all'indirizzo `076Dh` e ha sostituito `@DATA` con quel valore nei byte dell'eseguibile.

### Perché `076Dh` e non `076Ah`?

CS = `076Ah` mentre @DATA = `076Dh` perché i segmenti sono **separati e adiacenti** in memoria:

```
Memoria (indirizzi fisici):

┌──────────────────┐
│   PSP (256B)     │  ← DOS lo crea automaticamente
├──────────────────┤ ← 076Ah × 10h = 07A60h
│   .CODE          │  ← CS = 076Ah
│   (istruzioni)   │
├──────────────────┤ ← 076Dh × 10h = 07AD0h
│   .DATA          │  ← DS punterà qui dopo MOV DS, AX
│  n1, n2, msg...  │
├──────────────────┤
│   .STACK         │
└──────────────────┘
```

Ecco perché la prima cosa che fa `main` è caricare `076Dh` in DS: così il processore sa dove trovare le variabili!